Context-Aware Linguistic Steganography: A Generative Workflow using Large Language Models

NASOUH ALOLABI, Higher Institute for Applied Sciences and Technology, Syria
RIAD SONBOL, Higher Institute for Applied Sciences and Technology, Syria

This paper proposes a context-aware **black-box** linguistic steganography workflow for threaded discussions that carries the payload through a *selection channel* rather than token-level manipulation. The sender reconstructs a shared context corpus, selects both a reply target and a semantic angle, and uses a stochastic LLM to realize the choice as an ordinary visible reply. The receiver rebuilds supplied pre-publication context and recovers the selected choices; exact recovery depends on shared operational parameters, synchronized inputs, and the supplied snapshot boundary. The method avoids invisible control characters, but does not claim token-level or information-theoretic security. We also document a historical audited configuration comparison: 554 of 627 project-side attempts succeeded, while the baseline lane was invoked on 460 attempts and accepted and decode-verified 304. Those artifacts used audit-assisted project-side recovery and reused source posts, so their capacity and quality differences are configuration-level evidence rather than a current, decision-grade superiority claim.

Additional Key Words and Phrases: Linguistic Steganography, Large Language Models, LLMs, Natural Language Processing, NLP, Black-box Steganography, Semantic Embedding, Agentic Workflows, Imperceptibility

PREPRINT NOTE

This manuscript is a preprint dated August 8, 2026.

1 INTRODUCTION

Linguistic steganography focuses on the covert embedding of secret information within ostensibly innocuous textual carriers. Traditional methodologies, primarily predicated on lexical substitution and synonym manipulation [1], often struggle with the inherent low redundancy of natural language. Consequently, even minor statistical deviations can provide sufficient signal for modern automated detection systems [2–4]. The emergence of Large Language Models (LLMs) has catalyzed a paradigm shift, transitioning the field from text modification to de novo text generation [5–7]. By sampling directly from a model's learned probability distribution, these generative approaches theoretically achieve high levels of statistical imperceptibility.

However, existing LLM-based steganographic systems remain constrained by a fundamental trade-off between statistical security and perceptual naturalness [5–7]. While a model may output grammatically and statistically fluent sentences, these often lack the deep semantic coherence required for specific communicative contexts, such as threaded social media discussions [8–10]. A comment that is technically flawless yet pragmatically irrelevant to the ongoing discourse serves as a conspicuous anomaly under human or semantic-level scrutiny.

To address these challenges, we introduce a context-aware steganographic framework that elevates the embedding domain from the lexical level (individual tokens) to the semantic level (conversational decisions). Instead of manipulating token distributions, our scheme carries the payload through a *selection channel*: bits map to (i) the index of the reply target within a deterministically flattened comment tree, and (ii) the index of a context-conditioned *Angle* within a shared angle set that both parties reconstruct deterministically from the public thread and reproducible research. A

Authors' addresses: Nasouh AlOlabi, Higher Institute for Applied Sciences and Technology, Damascus, Syria; Riad Sonbol, Higher Institute for Applied Sciences and Technology, Damascus, Syria.

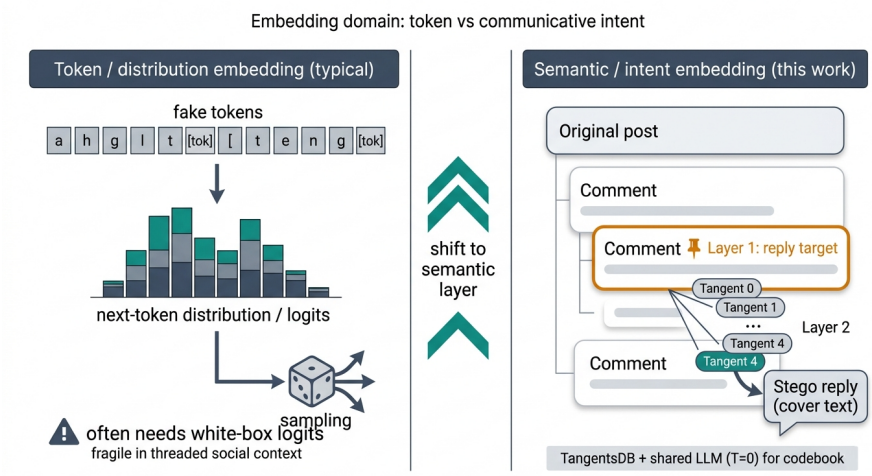

Fig. 1. Conceptual contrast between token-level embedding and our two-layer semantic embedding over a threaded discussion.

language model then generates an ordinary visible reply that serves as a *decodable witness* of the selected Angle; we do not rely on invisible characters or control Unicode. A sender-side verification loop closes the system by simulating receiver decoding before publication. Figure 1 contrasts token-level embedding with our two-layer selection-channel embedding over a threaded discussion.

The primary contributions of this paper are:

- A two-layer, selection-channel embedding architecture for threaded discussions that encodes bits into reply targeting and Angle selection, with visible text serving as a decodable witness rather than a hidden carrier.
- A deterministic reconstruction pipeline (thread + reproducible research) that enables sender/receiver parity without white-box model access or side-channel metadata exchange.
- A closed-loop verification mechanism that simulates decoding pre-publication and retries synthesis to improve recoverability under stochastic generation.
- An audit of a historical configuration comparison against the official zero-shot generative linguistic steganography baseline, with explicit attempt accounting, failure categories, capacity, perplexity, rescaled MATTR, length, and post-clustered caveats; it is not a current decision-grade superiority claim.

2 BACKGROUND

Among the classical concerns of information security—protecting content through encryption, preserving user privacy, and concealing the existence of communication itself—this work is concerned with the last. **Steganography** differs from encryption in a fundamental way: rather than making a message unreadable, it makes the message invisible by embedding it inside an ordinary carrier, such as an image or a piece of text, so that an observer has no reason to suspect anything is hidden. **Imperceptibility** is therefore the primary objective, not confidentiality per se.

Text is a particularly challenging carrier due to its low redundancy and strict semantic constraints. The classical "Prisoners' Problem" [11] illustrates the goal: two parties, Alice and Bob, must exchange hidden information without alerting a watchful adversary.

Textual steganography methods are typically divided into **format-based** approaches, which exploit layout or structural features, and **linguistic-based** approaches, which modify linguistic form. Within the latter, early techniques such as **synonym substitution** embed bits by altering lexical choices, but suffer from low capacity and high detectability.

More formally, generative linguistic steganography operates through an **embedding function** f_{emb} that establishes a cryptographic and linguistic mapping to hide secret messages within generated text. The algorithm takes as inputs a language model $\mathcal{L}$ and a secret message m (represented as a uniformly distributed bitstream). At each generation step, rather than sampling freely from the language model's conditional distribution p_{LM}, the embedding function f_{emb} **restricts or modifies** the token selection process according to the message bits being embedded. This specialized process is known as **steganographic sampling**, producing a steganographic text (stegotext) y. The security of the system depends on minimizing the divergence between the original language model distribution p_{LM} and the modified distribution q induced by f_{emb} [12].

Traditional linguistic approaches offer limited embedding capacity and often leave statistical artifacts. Advances in deep learning and **Large Language Models (LLMs)** now enable generative methods that achieve higher text quality and more secure embedding. Evaluating such systems requires several dimensions of imperceptibility: **perceptual** (human naturalness), **statistical** (distributional similarity to natural text), and **cognitive** (semantic and contextual fidelity) [13].

LLM use in steganography also introduces new sources of detectability. **Hallucination**—where a model generates content that is factually unsupported, semantically inconsistent, or poorly grounded in the preceding context—is one such source. In ordinary text generation, hallucination is mainly a reliability concern. In steganographic generation, it becomes a security concern as well: even a fluent sentence can expose a hidden channel if it contains an unsupported claim, an abrupt topic shift, or a contradiction with the surrounding context. Hallucination therefore threatens **cognitive imperceptibility**, since generated text must remain contextually plausible and communicatively coherent, not merely surface-fluent.

Distinct from hallucination, the **PSIC effect** (Perceptual-Statistical Imperceptibility Conflict) [2] captures a different and arguably more fundamental tension in generative linguistic steganography. Text that is optimized for human fluency is not necessarily statistically indistinguishable from natural human text. For example, selecting only high-probability tokens can produce sentences with low perplexity and strong local readability, but it may also make the generated distribution overly smooth, regular, or concentrated compared with real human writing. Such text can appear natural to readers while remaining detectable to statistical steganalyzers [2].

The PSIC effect therefore shows that **perceptual imperceptibility** and **statistical imperceptibility** are related but not identical goals. Human-written text often contains variation, irregularity, and locally unexpected word choices. By contrast, overly fluent model-generated text may lack this natural variance. Increasing the embedding rate can sometimes force the sampler to select from a broader range of tokens, which may better approximate the diversity of human text; however, this same process can also reduce readability or semantic coherence if the token choices become poorly aligned with the context. Thus, higher capacity does not automatically imply better security. Secure generative steganography requires balancing embedding rate, fluency, distributional similarity, and contextual consistency.

2.1 Large Language Models as Text Generators

Large Language Models (LLMs) are autoregressive, generative systems based on the Transformer architecture [14] that approximate high-dimensional distributions over natural-language sequences [3, 15]. Given a prefix, an LLM emits a probability vector over the vocabulary; the next token is sampled from this vector and appended to the prefix, and the

process repeats until a stopping criterion is met. During pre-training, billions of parameters are tuned on large text corpora so that the model's predictive distribution approximates patterns in the training data [16]. As a consequence, modern LLMs can produce text with high fluency, coherence, and stylistic control [17]. The learned representations capture stylistic and semantic regularities that generalize across domains, enabling applications requiring nuanced linguistic mimicry [18].

2.2 Large Language Models in Generative Linguistic Steganography

Because text produced by modern LLMs can closely resemble ordinary human-authored text, these models provide effective engines for generative text steganography. Rather than editing an existing cover, a growing line of work treats the model itself as a steganographic sampler: the hidden message steers generation, so the stego text is drawn from a realistic communication distribution from the start. The public availability of high-quality models, together with the efficiency of sampling-based embedding, has made this route more practical than earlier designs.

LLMs like **GPT-2** [15], **LLaMA** [19], and **Baichuan2** [20] are commonly used as basic generative models for steganography. Most such schemes couple the language model with a steganographic code that ties message bits to token choices: at each generation step, the bits to be hidden determine which candidate token is emitted from the model's next-token distribution. However, traditional "white-box" methods necessitate sharing the exact language model and training vocabulary, which limits fluency, logic, and diversity compared to texts generated by modern black-box LLM APIs. These methods inevitably alter sampling probability distributions when embedding messages, posing security risks [7]. **Black-box approaches** avoid these limitations by leveraging state-of-the-art hosted models to deliver better text quality, faster generation speeds, and minimal local resource requirements. However, this paradigm trades fine-grained control over internal sampling for these practical advantages. Conversely, **white-box access** enables precise parameter control and stronger security guarantees, but demands substantial computational resources, engineering effort, and higher latency—raising deployment barriers. This trade-off between access paradigms is central to evaluating the robustness and applicability of modern linguistic steganography, and it directly motivates the black-box design adopted in Section 4.

New approaches, such as **LLM-Stega** [7], explore black-box generative text steganography using the user interfaces (UIs) of LLMs, which removes any need to observe internal sampling distributions. Secret bits are encrypted and mapped onto a prearranged keyword set that steers generation, and candidate outputs from which the message cannot be recovered are discarded via reject sampling, preserving both extraction accuracy and semantic quality [7].

Another framework, **Co-Stega**, leverages LLMs to address the challenge of low capacity in social media. It expands the space available for hiding messages through context retrieval and uses purpose-built prompts to raise the entropy of the generated replies, which in turn enlarges the embedding capacity while keeping the output fluent and topically relevant [8].

Zero-shot linguistic steganography instead relies on in-context learning: covertext samples are supplied in the prompt, and the stego text is produced within a question–answer (QA) exchange, so the output reads as a natural reply rather than as free-standing generated text [6].

The increasing popularity of deep generative models has made it feasible for provably secure steganography to be applied in real-world scenarios, as they fulfill requirements for perfect samplers and explicit data distributions [3, 21, 22].

LLM-based steganographic methods are typically evaluated on two primary axes: imperceptibility (perceptual, statistical, and cognitive measures) and embedding capacity (e.g., bits per token or bits per word). Imperceptibility

evaluations may include automatic metrics (PPL, Distinct-n, KL/JSD) as well as human judgements; embedding capacity is usually reported as bits/token or overall embedding rate. Section 3 surveys how the literature applies, evaluates, and constrains these methods in more detail.

3 RELATED WORK

Majeed et al. (2021) [23] surveyed pre-LLM text steganography techniques, predating the current transformer era, emphasizing synonym replacement, spacing manipulation, and coding schemes such as Huffman coding. The methods of that generation often relied on context-free grammars or Markov chains and tended to produce syntactically correct but semantically incoherent cover text. Setiadi et al. (2025) [24] examine more recent AI-powered data hiding across image, linguistic, and 3D mesh carriers (2021–2025), observing that LLMs have "revitalized" linguistic steganography through methods built on GPT-2 [25], GPT-3 [16], LLaMA2 [26], and Baichuan2 [27]. On the detection side, Wang et al. [4] review deep-learning linguistic steganalysis and document how detection techniques have evolved alongside generative hiding methods.

These works provide valuable coverage of text steganography, but they are best described as narrative surveys rather than exhaustive systematic literature reviews, and none of them foregrounds *contextual compatibility*—the degree to which generated cover text fits its surrounding discourse, task, and conversational setting—as a central evaluative criterion for LLM-based methods. Crucially, the field has evolved from "statistical vector embedding" (Word2Vec, GloVe) to "language-model vector embedding" that exploits BERT-scale transformers and higher-dimensional semantic spaces. This creates a methodological gap: no systematic review comprehensively maps how large-scale transformers have redefined text steganography along this dimension. Modern advances extend beyond naive generation to sophisticated Controllable Text Generation (CTG) frameworks [28]. These employ Variational Autoencoders (VAEs) to model latent features and diffusion models to inject randomness, mitigating spurious associations between secrets and control conditions. Contemporary approaches leverage prompt learning and prefix tuning, enabling efficient model customization without costly full fine-tuning.

Defensive strategies must evolve accordingly. Traditional steganalysis, premised on hand-crafted statistical features, falters against generative steganography's high statistical concealment. Current research must confront "stegomalware"—attacks that conceal command-and-control communications within innocuous digital media [29].

To close this gap, the remainder of this section presents a structured synthesis of 31 selected primary studies on LLM-based linguistic steganography published between 2020 and 2025, organized around the state of the published literature, its application domains, its evaluation practices, its patterns of external-knowledge integration, and its recurring limitations and trade-offs. Table 1 summarizes how this synthesis differs from the three prior surveys discussed above.

3.1 State of the Published Literature

The 31 studies span 2020 through 2025, with 10 publications falling in 2020–2023 and 21 in 2024–2025 alone—a sharp acceleration that coincides with the wider availability of large-scale models. Widely used model families—GPT, GPT-2, and LLaMA [15, 16, 19]—appear across both periods, though proprietary systems grow more common in the later studies as black-box access becomes a viable design choice. Across the reviewed corpus, open-weight and research-accessible models appear more frequently than closed commercial systems, reflecting the practical importance of reproducibility and controllable experimentation.

The field has also evolved from earlier white-box token-sampling schemes toward a broader design space that includes hybrid and black-box methods. The reviewed studies now span generative schemes that write stego text from

Table 1. Comparison of the literature synthesis in this section with prior surveys of text/linguistic steganography.

Survey	Period covered	Methodology	Carrier scope	LLM-era coverage	Contextual-compatibility focus
Majeed et al. [23] (2021)	Pre-transformer	Narrative	Text-only (synonym, spacing, coding)	None	No
Setiadi et al. [24] (2025)	2021–2025	Narrative	Multi-carrier (image, linguistic, 3D mesh)	Yes	No
Wang et al. [4] (2023)	Deep-learning era	Narrative	Detection-side (linguistic steganalysis)	Partial	No
This synthesis	2020–2025	Structured synthesis	Text-only (LLM-based)	Yes	Yes

Table 2. Representative model families across the reviewed studies. Categories are non-exclusive, so totals exceed 100%.

Model Type (Studies)	Models and Representative Works
Open-weight Models (25/31, 80.6%)	GPT-2 [3, 9, 21, 36], LLaMA/LLaMA2 [6, 8, 22, 37], BERT [1, 10, 13, 28, 38–42], OPT [43], BART [1, 5], Qwen [33, 44]
Proprietary Models (7/31, 22.6%)	GPT-3.5/4, ChatGPT [7, 30–32, 45, 46]
Custom or Task-Specific Architectures (5/31, 16.1%)	From-scratch or task-specific models [2, 35, 39, 40, 43]

scratch [2, 3, 12, 21], rewriting systems that paraphrase existing covers [5], black-box pipelines that treat the model as an opaque API [7, 30–32], zero-shot protocols driven primarily by prompting [6], context-aware frameworks that retrieve or constrain external information to increase semantic plausibility or payload [8, 9, 33], and methods that emphasize stronger security, robustness, or token-level quality control [3, 21, 34, 35].

Table 2 shows how the reviewed studies distribute across model families. This table is best read as a synthesis of methodological tendencies rather than a strict one-study-one-category classification: many papers combine multiple model families, so the counts overlap and the percentages do not sum to 100%. Open-weight models account for the largest share because they support reproducible experiments and direct control over sampling or decoding, with GPT-2 and LLaMA-family systems appearing especially often. BERT-based methods also recur frequently, but usually as masked-language or scoring components rather than as the sole generation backbone. Proprietary systems such as GPT-3.5/4 are mainly used in black-box or prompt-driven settings, while custom or task-specific architectures appear less often but remain important for studies that prioritize provable security, reversible encoding, or tightly controlled payload design.

3.2 Application Domains

The reviewed studies show that **covert communication** remains the dominant application of LLM-based linguistic steganography. In this setting, secret information is embedded into seemingly benign text so that the message can pass through ordinary communication channels with minimal suspicion. LLM-based methods improve this use case by producing more fluent and contextually plausible outputs than earlier rule-based or local-substitution systems [2, 3, 7].

Within this application space, the reviewed papers explore several operational variants. Some methods generate stego text from scratch using a shared model or sampling strategy [3, 21]. Others rely on paraphrasing or rewriting so that a pre-existing cover text can be transformed while preserving its apparent meaning [5]. Recent black-box

Table 3. Application categories across the reviewed studies. Categories are non-exclusive.

Primary Application	Variant / Category	Representative Studies	Total Count
Covert Communication	Generation-based (from scratch)	[2, 3, 21, 22, 28, 34–36, 39, 41, 42]	11
	Paraphrasing, Rewriting, or Editing	[5, 38]	2
	Black-box and Prompt-based	[6, 7, 30–32, 37]	6
Context-Aware Systems	Domain, Emotion, or Social Adaptation	[8–10, 13, 33, 44]	6
Watermarking	Content Attribution (Boundary Cases)	[1, 40, 43, 45–47]	6

and prompt-based approaches such as **LLM-Stega**, **DeepStego**, and multi-criteria linguistic optimization show that covert communication can also be implemented through commercial or hosted interfaces without direct access to token probabilities, using prompt design, keyword sets, stylistic features, and reject-sampling style verification instead [7, 31, 32]. Context-aware systems such as **Co-Stega**, **Hi-Stega**, and emotionally controllable steganography further adapt the generated text to specific domains, topics, entities, emotional tones, or social settings in order to improve plausibility and increase effective payload [8, 9, 33]. This last variant—adapting generation to the surrounding domain and discourse rather than relying on generic fluency—is the closest precedent in the literature to the context-aware, thread-level embedding developed in Section 4.

Although adjacent topics such as watermarking, authorship attribution, or prompt-hiding attacks are relevant to the broader field of information hiding, they remain analytically distinct from covert payload transmission. The 2025 watermarking studies are therefore treated as boundary cases: useful for understanding robustness, semantic preservation, and detection methods, but not interchangeable with steganographic communication systems [47].

A comprehensive breakdown of how the reviewed studies are distributed across these primary application areas and variants is summarized in Table 3.

3.3 Evaluation Practices in the Literature

Performance evaluation for LLM-based steganography relies on three main categories of metrics, but reporting standards vary widely across studies. This variation limits direct comparison and motivates more consistent evaluation protocols; it also directly informs the choice of metrics used for this paper's own evaluation in Section 6.

3.3.1 Perplexity. Perplexity is an imperceptibility metric for fluency, where lower values generally indicate more natural text [48]. However, its usefulness for language model evaluation is limited. It mainly reflects internal consistency rather than factuality or reasoning, so fluent but implausible statements can still receive low perplexity scores [49]. It is also sensitive to text length, with short sequences often receiving inflated scores even when they are highly fluent [50]. In addition, perplexity is not directly comparable across models unless tokenization and vocabulary are aligned, since these design choices affect the score itself [51]. Fang et al. [50] further argue that token-averaged likelihood can under-represent performance on important low-probability tokens, while Wang et al. [49] note that repetition can sometimes lower perplexity in misleading ways. Taken together, these issues suggest that perplexity should be used cautiously and alongside other measures when assessing steganographic text quality.

3.3.2 Statistical Divergence Metrics. Kullback-Leibler Divergence (KLD) [52] and Jensen-Shannon Divergence (JSD) are information-theoretic metrics used to evaluate steganographic security. KLD quantifies information loss by measuring the relative entropy between cover and stego distributions, serving as the theoretical standard for security modeling

despite being asymmetric and failing as a strict distance measure. JSD improves upon this as a symmetric, bounded variant that measures how far each distribution lies from their average, providing a more stable basis for formulating statistical imperceptibility bounds—particularly when language models approximate human text distributions. Together, these two metrics attempt to capture how closely steganographic outputs mimic legitimate communication channels.

However, real-world application reveals reliability limits, most notably the PSIC effect [2]. KLD and JSD scores can diverge from human judgment as statistical optimization progresses: methods with favorable divergence scores may still produce low-quality text that appears suspicious to human observers. This discrepancy is also dataset dependent—the same method can yield different divergence values on different corpora at equivalent embedding rates (e.g., KLDs of 19.507 on IMDB versus 8.295 on Twitter for VAE-Stega [2]), making cross-paper comparison unreliable. In addition, studies use different formulas, feature spaces, and measurement scales, including latent BERT features versus direct word distributions. Meteor [3], for example, is associated with reported KLD values ranging from 0.045 in one setting to 7.491–11.845 in others. Consequently, these metrics capture distributional distance but not perceptual naturalness, so they should be paired with semantic and human-centered measures.

3.3.3 Capacity Metrics. Capacity is judged by four metrics:

- **Bits per Token (BPT):**

$$\text{BPT} = \frac{\text{Total Secret Bits}}{\text{Total Tokens}} \tag{1}$$

- **Bits per Word (BPW):**

$$\text{BPW} = \frac{\text{Total Secret Bits}}{\text{Total Words}} \tag{2}$$

- **Embedding Rate (ER) [53]:** Average density of hidden information per textual unit

$$\text{ER} = \frac{1}{N}\sum_{i=1}^{N} \text{bits}_i \tag{3}$$

  where N is the number of textual units (words, tokens, or sentences) and bits_i is the number of bits embedded in the i-th unit. In some fixed-payload schemes, the total number of embedded bits is predetermined for each stego text and does not depend on its length; for example, LLM-Stega can assign a constant payload to every generated stego sentence, so the embedding rate can also be expressed as $\text{ER}_{\text{fixed}} = B_0$, where B_0 is the fixed number of bits embedded in each stego text.

- **Utilization Rate (UR):**

$$H = -\sum_{x \in X} P(x) \log_2 P(x) \tag{4}$$

$$\text{UR} = \left(\frac{\text{Actual Bits Embedded}}{H}\right) \times 100\% \tag{5}$$

These quantities describe how densely a secret is embedded, but they are affected by systematic biases that limit cross-system comparison, summarized in Table 4.

Tokenization differences make "1 BPT" from one paper difficult to compare with "1 BPT" from another when the studies use different tokenizers. The PSIC effect shows that higher density can hurt human fluency yet help statistical evasion. Model frequency preferences shrink the real alphabet to high-probability tokens, so naive entropy limits overstate usable space. Ambiguous reporting—practical vs. effective payload, or bit length vs. stego length—can make capacity claims appear stronger than they are. Finally, BPW/BPT ignore semantics, so high values can be reported even when the resulting text is suspicious or incoherent. Recent works reveal additional distortions: loop-count

Table 4. Five primary bias categories affecting capacity metrics in steganographic evaluation.

Bias Category	Core Problem	Critical Implication
Tokenization Inconsistencies	Metrics depend entirely on specific tokenizers (e.g., GPT-2 BPE vs. word-level)	Direct comparisons across papers become meaningless when tokenization strategies differ
The PSIC Effect	Conflicts between imperceptibility and statistical security are ignored	High capacity may degrade human fluency while paradoxically improving detection resistance
Model Training Bias	Utilization Rate calculations assume uniform token availability	Actual hiding space is smaller than theoretical entropy due to model frequency preferences
Reporting Ambiguities	No standard definition of "capacity" across systems	Practical payload vs. effective payload distinctions create misleading efficiency claims
Context Blindness	Density metrics treat text as neutral bit containers	Semantic incoherence constitutes a security failure that BPW/BPT fails to penalize

Table 5. Representative capacity calculations reported by selected methods. Full generated passages are omitted because they do not support direct cross-study comparison.

Method	Reported Capacity	Encoding Basis	Comparability Note
Meteor	1280 bits	160-byte payload × 8 bits	Payload total; depends on the selected message length.
LLM-Stega	64 bits per sentence	Fixed semantic-slot mapping	Sentence-level capacity; not directly comparable with BPW or BPT.
GCStego	16 bits per word	Fixed word-level mapping	Mapping-specific rate under the study's candidate construction.
ParaLS	3 bits	Binary lexical watermark	Example payload rather than a normalized embedding rate.
Zero-shot	Token-level BPW/capacity	Generated-token coding	Capacity depends on the representation, model, and prompt protocol; not directly comparable with selection bits.
Discop	968 bits over 200 tokens	4.84 bits per token at $p = 0.95$	Token-level rate tied to the model, tokenizer, and truncation setting.

overhead, dictionary-size caps, baseline-dependent "improvements," watermarking goals that invert the desired signal, conversational filler that dilutes BPW, and nonlinear ER curves that make any single threshold difficult to interpret. The added 2025 studies reinforce this point: DeepStego emphasizes high capacity and recovery accuracy, and multi-criteria optimization reports reliable stylistic decoding [31, 32], but these results also make cross-paper comparison harder because the reported gains depend on different baselines, prompts, datasets, and decoding assumptions.

To contextualize theoretical capacity metrics, Table 5 summarizes representative embedding capacities across the reviewed methods. Capacity calculations vary significantly based on the encoding mechanism. **Fixed-length mapping** techniques, such as **LLM-Stega** and **GCStego**, derive capacity from rigid keyword or token mappings—64 bits per sentence and 16 bits per word, respectively. **ParaLS** operates similarly but on a smaller scale, encoding a simple 3-bit binary watermark. In contrast, **payload-dependent** methods scale with the cover text or secret message length: **Meteor's** capacity of 1280 bits corresponds to an embedded payload of 160 bytes, **Zero-shot** capacity is directly tied to the Base64 representation of the secret bitstream, and **Discop** uses a statistical approach that, with a truncation parameter of $p = 0.95$, achieves an entropy rate of 4.84 bits per token, yielding 968 bits over a 200-token sequence.

3.3.4 Security Metrics. The sources evaluate security and reliability through the following integrated metrics:

- **Detection Accuracy (Acc) / Error Rate (PE):** Measures a classifier's ability to distinguish between cover and stego text. An accuracy of 50% (or PE of 50%) is the gold standard, indicating the stego text is statistically identical to normal text.
- **F1 Score:** The harmonic mean of precision and recall, used to verify detection reliability, especially when datasets are balanced.
- **Robustness (Attack Resistance):** Evaluated via the Mean Impact of Attack (mIOA) or bit accuracy after removal attempts such as denoising auto-encoding, word substitution, or sentence deletion.

Table 6. Frequency of evaluation metric categories in the reviewed studies. Categories are non-exclusive.

Dimension	Metric Category	Studies	Main Comparability Constraint
Imperceptibility	Perplexity	13	Model, tokenizer, corpus, and sequence length
Imperceptibility	Statistical divergence (KLD/JSD)	10	Estimated distributions and sampling settings
Imperceptibility	Semantic similarity	13	Reference construction and metric choice
Imperceptibility	Human evaluation	1	Protocol, annotator population, and sample size
Security	Detection accuracy or F1	21	Detector, class balance, and train/test data
Capacity	BPW or BPT	27	Tokenization and payload accounting
Robustness	Recovery or attack resilience	5	Attack type, strength, and evaluation protocol
Efficiency	Generation or extraction time	5	Hardware, model size, and implementation

- **False Positive Rate (FPR):** Measures how often human or normal model-generated texts are incorrectly flagged as containing secret messages.

Recent robustness-oriented work broadens this evaluation set. Position-agnostic generation tests perturbation tolerance across multiple attack types, active-attack repair methods evaluate message integrity after tampering, and contrastive semantic watermarking reports F1 under rewriting and substitution attacks [34, 35, 47]. This shift suggests that future evaluations should report not only clean-channel capacity and fluency, but also degradation curves under realistic editing.

3.3.5 Evaluation Challenges and Gaps. The broad range of evaluation metrics across the reviewed studies is summarized in Table 6, which categorizes metrics by type and quantifies their usage frequency. The numbers in that table reveal several persistent gaps. Shared benchmarks are rare: only around 20% of studies use common datasets, which makes cross-paper comparison difficult even when the same metric name appears in both papers. Human evaluation was reported in just one of the 31 reviewed studies—a surprisingly low share given how central perceptual plausibility is to the field's core claims. Robustness testing is similarly sparse, absent in roughly 60% of studies. Many papers also evaluate along only one or two metric dimensions, leaving imperceptibility, security, and capacity to be inferred from partial evidence rather than directly measured together. Taken together, these gaps mean the strongest conclusions available from the reviewed corpus are qualitative rather than quantitative—a limitation this paper addresses directly by reporting naturalness, distributional divergence, and operational reliability jointly for the same runs (Section 6).

3.4 Integration of External Knowledge Sources

External knowledge integration is a recurring design pattern in the reviewed literature. Across the surveyed studies, external information serves three main purposes: improving contextual plausibility, maintaining topic consistency, and—in some cases—expanding the effective space available for embedding. Table 7 summarizes the main knowledge-integration patterns and their roles.

Rather than relying only on the internal prior of the language model, several recent systems inject external guidance during generation. Retrieved posts, headlines, or comments can provide a realistic discourse context for the cover text [9]. Knowledge-graph triples can constrain the content that should appear in the output, improving semantic consistency over longer spans [37]. Named entities and sentiment labels add a finer-grained control layer when the goal

Table 7. Representative patterns of external knowledge integration.

Knowledge Type	Typical Role	Primary Benefit	Examples
Semantic Resources	Topic, entity, and emotion guidance	Better semantic control	Co-Stega [8], knowledge-graph guided methods [37, 39], emotionally controllable steganography [33]
Domain Corpora	Domain adaptation	Better stylistic fit	FREmax [36], specialized datasets
Prompt Engineering	In-context control	Better task alignment	Zero-shot [6], DeepStego [31], multi-criteria optimization [32]
Context Retrieval	Evidence or example retrieval	Higher contextual plausibility	Co-Stega [8], retrieval-augmented pipelines [9]

is emotional or conversational consistency [33]. Frequency-based external statistics can also be used to steer generation toward distributions that better match the target channel [36].

Another recurring pattern is the use of domain-specific corpora. Fine-tuning or conditioning on domain data helps align the generated text with the vocabulary, stylistic conventions, and topical expectations of the intended communication channel. This pattern appears in systems such as VAE-Stega [2], Meteor [3], Hi-Stega [9], FREmax [36], Rewriting-Stego [5], and Summarization-Stego [10]. When additional training is not possible, black-box methods move this domain adaptation into the prompt, style specification, or retrieval stage. Zero-shot approaches provide examples directly in the context window, LLM-Stega frames the generation request with topic-constrained prompts, DeepStego uses prompts and synonym guidance, multi-criteria optimization encodes through stylistic controls, and Co-Stega retrieves recent posts to anchor the generated reply in a plausible local context [6–8, 31, 32]—the same retrieve-then-anchor strategy that underlies the deterministic research stage described in Section 4.3.

Table 8 provides a more detailed breakdown of how external knowledge sources are integrated across the reviewed studies.

These five patterns differ primarily in how directly they constrain the generation process. Structured knowledge approaches [37, 39] use external knowledge graphs or graph attention networks to enforce long-range semantic consistency—a stronger form of control than prompting alone, but one that requires the knowledge resource to exist and be aligned with the target domain. Information retrieval frameworks [8, 9] pull contextually relevant material from external sources at runtime, grounding the cover text in real discourse without requiring model retraining. Linguistic resource methods [1] rely on synonym databases and paraphrase corpora to constrain lexical choice while preserving meaning—a lighter form of intervention that nonetheless depends on the quality and coverage of the external resource. External model feedback is the most common pattern, appearing in seven of the 31 reviewed studies: auxiliary models such as BERT serve as discriminators or soft scorers to assess naturalness or security during generation [41, 43]. Finally, shared side information methods [3, 6] use public models or auxiliary history to synchronize probability distributions between sender and receiver without explicitly passing knowledge through the channel, trading interpretability for coordination efficiency—the same trade-off that motivates the shared, deterministic search pipeline in Section 4.3.

3.5 Limitations and Trade-offs

Across the surveyed studies, the main constraints recur around imperceptibility, effective payload, semantic coherence, tokenization reliability, deployment assumptions, and attack robustness.

3.5.1 The PSIC Effect Revisited. Generative steganography faces an intrinsic tension: optimizing for perceptual fidelity degrades statistical security. Aggressive perplexity minimization produces unnaturally "sharp" distributions that deviate

Table 8. Comprehensive overview of external knowledge integration patterns in LLM-based steganography.

Approach	Variant / Category	Representative Studies	Count
Structured Knowledge Integration	Knowledge Graphs (KG) & Graph Attention Networks	Semantic controllable steganography via knowledge graph [37], joint linguistic steganography with BERT and graph attention networks [39]	2
	Information Retrieval (IR): Retrieval-Augmented Generation (RAG) & Corpus Search	Hi-Stega [9], Co-Stega [8], emotionally controllable text steganography [33]	3
Linguistic Resource Integration	Paraphrase DBs, Synonym DBs & Dependency Datasets	Paraphraser-based lexical substitution [1], DeepStego [31], post-hoc watermarking of black-box LLM text [46], group-chat imperceptible text steganography [44]	4
	External Statistical Analysis: Character/Token Frequency Distributions	FREmax [36]	1
External Model Feedback	Auxiliary Scorer, Critic & Discriminator Models	Rewriting-Stego [5], principled natural language watermarking [40], CPG-LS [41], controllable semantic steganography via summarization [10], ALiSa [42], DeepTextMark [43], contrastive semantic watermarking [47]	7
Shared Knowledge / Side Information	Shared Auxiliary Datasets & Public Contexts	Meteor [3], zero-shot generative linguistic steganography [6], reversible data hiding via masked language modeling [38], natural language steganography by ChatGPT [30]	4

from high-variance human writing, increasing detection vulnerability. Higher embedding rates force selection of lower-probability tokens, degrading text quality but expanding the candidate pool to enhance anti-steganalysis resistance. At elevated capacities, output distributions asymptotically approach natural language's high-entropy chaos, camouflaging signals within legitimate statistical noise [2].

3.5.2 Attack Vulnerability and Security Concerns. Current techniques exhibit a substantial attack surface across multiple vectors, including word-level deletions and lexical substitutions that disrupt dependency parsing. Polishing and re-translation attacks progressively erase watermarks; at high intensities, these structural changes can reduce detection performance to an F1 score of 0.5, rendering the watermark undetectable. Furthermore, adversarial machine learning enables the extraction of secret model parameters from black-box systems, potentially compromising integrity entirely. Recent work responds by moving robustness into the embedding design itself: position-agnostic generation avoids dependence on fixed token locations, active-attack repair models attempt to recover damaged stegotext, and contrastive semantic watermarking tests resilience under rewriting and substitution [34, 35, 47]. To mitigate distributional risks, Variational Auto-Encoders are also employed to learn the statistical distribution of normal text, aiming to reduce the KL divergence between cover and stego distributions and resolve the conflict between perceptual and statistical imperceptibility [46].

3.5.3 Capacity and Contextual Constraints. Short, low-entropy contexts severely constrain embedding capacity. Social media posts and technical documents offer minimal redundancy for hiding [8], while semantic preservation requirements compete directly with information embedding objectives. Brief texts further lack sufficient contextual support for effective steganographic operations [13]. Newer approaches attempt to recover some capacity through synonym guidance and stylistic feature control, but these gains remain tied to the selected prompt, cover text, or candidate-token space [31, 32].

Early generative methods also produced fluent but semantically arbitrary text, violating cognitive imperceptibility [13]. When generation is conditioned only on local token history, the output may drift away from the source topic or conversational logic, which is especially suspicious in low-entropy channels such as social media replies. This contextual drift worsens when methods must keep generating until an arbitrarily large payload is fully embedded: texts become unnaturally long, coherence decays over time, and practical payload efficiency drops even when reported bit capacity appears high [30, 37, 41, 45].

Our paired benchmark against the zero-shot generative linguistic steganography baseline [6] is reported in Section 6. In the tested configuration, the baseline had higher recoverable capacity and lower GPT-2 perplexity, while our method produced longer replies and slightly higher MATTR (rescaled lexical diversity). The comparison is conditional on accepted outputs and is therefore accompanied by complete attempt and failure counts; it does not establish a security advantage for either method.

3.5.4 Segmentation and Tokenization Barriers. Subword tokenization introduces critical extraction ambiguities that are fundamental to white-box steganographic methods. Byte-pair encoding splits words unpredictably, creating multiple valid segmentations that corrupt message boundaries and cause decoding failures. These ambiguities impose hard capacity caps regardless of algorithmic sophistication—SparSamp suffers accuracy degradation from token ambiguity, while ShiMer cannot effectively boost entropy due to tokenization constraints [22].

This segmentation ambiguity persists even when sender and receiver employ identical LLMs and tokenizers. Because modern LLMs decompose words into subword units, and vocabularies frequently contain both complete words and their constituent components, any given text string admits multiple valid token representations. This multiplicity is compounded by the detokenization-retokenization gap: senders must convert tokens to plain text to conceal their transmission, while receivers must subsequently retokenize this text to recover hidden messages. Despite shared tokenization rules, receivers may select different token sequences than senders intended—a sender might encode a message using two separate tokens (“any” and “thing”), only for the receiver to retokenize the surface string as the single token “anything”. Because LLMs operate autoregressively, a single divergent token choice alters the probability distribution for all subsequent predictions, causing total decoding failure for the remainder of the message. To address this white-box-specific challenge, researchers propose SyncPool, a mechanism that aggregates ambiguous tokens into pools and employs synchronized random number generators to ensure both parties select identical tokens [22]. Because this work embeds payload through reply targeting and angle selection rather than through raw token identity (Section 4.6.1, Section 4.6.2), it sidesteps the detokenization-retokenization gap entirely: the sender and receiver only need to agree on which discrete angle or reply target was selected, not on an exact token sequence.

3.5.5 White-box versus Black-box Trade-offs. The architectural choice between white-box and black-box access paradigms involves fundamental trade-offs. Across the reviewed corpus, only a minority of studies operate primarily through black-box or prompt-driven access (Table 3), while the majority employ white-box or hybrid methods. White-box methods typically assume access to internal language-model components such as token probabilities, decoding

states, vocabularies, or model parameters, and some approaches additionally require expensive fine-tuning or retraining procedures to align the model distribution with steganographic objectives. While these methods can achieve strong statistical imperceptibility through distribution-aware generation (e.g., VAE-Stega and arithmetic-coding-based methods), they suffer from reduced accessibility, high computational cost, and deployment complexity.

In contrast, black-box approaches operate through standard LLM APIs or user interfaces without requiring direct access to model internals or retraining, substantially improving practicality and usability. However, the distinction is primarily about *model access*, not the absence of probabilistic coordination, as several black-box approaches still rely on shared probabilistic mechanisms, semantic distributions, or synchronized auxiliary structures between sender and receiver. Recent semantic black-box methods also prioritize robustness against paraphrasing, token perturbation, and noisy communication channels rather than purely token-level imperceptibility. Among the few peer-reviewed black-box methods currently available, LLM-Stega reports a capacity of 5.93 bpw with perplexity 165.76, illustrating that improved accessibility does not eliminate the PSIC trade-off between payload capacity, text quality, robustness, and security. Several recent systems further blur strict categorical boundaries by combining retrieval-, generation-, editing-, or substitution-based paradigms within hybrid steganographic frameworks.

3.5.6 Computational and Resource Constraints. Performance optimization conflicts directly with computational efficiency [3]. Superior results demand increased resources and memory, particularly for large models with external knowledge bases [21]. Real-time latency constraints limit optimization depth, while performance degradation emerges at operational scale. Variable length sampling, rejection sampling, active repair, and token-level quality control significantly increase encoding or decoding complexity, and LLM-based steganography requires 3x to 5x more time than prior methods [22]. The Meteor system exemplifies hardware constraints, running nearly 50× slower on mobile devices compared to GPU environments. LLM-specific deployment obstacles compound these costs: hallucinations and nonsensical continuations can corrupt the covert bitstream or create detectable artifacts, while lossy transmission, editing, or platform-side normalization can irreversibly damage extraction. As model quality improves, steganalysis systems also improve, producing an arms race in which better generation does not automatically translate into better security.

3.5.7 Critical Unresolved Challenges. Foundational gaps persist across the field: theoretical security guarantees remain unestablished, resilience to advanced attacks is limited, evaluation frameworks lack standardization, responsible development guidelines are absent, non-English performance lags substantially, and real-world deployment testing remains insufficient. Several studies also surface unresolved concerns around reversibility, bias, and misuse potential, indicating that robust engineering alone will not be enough without clearer security and governance models [1, 6, 38, 54].

Quantitative evidence underscores several of these constraints. The PSIC effect creates a trade-off where increasing the embedding rate (e.g., to ~5 bpw) strengthens anti-steganalysis but significantly degrades perceptual text quality [2]. Attack vulnerability is notable; for example, adaptive attacks can reduce bit accuracy to near-random levels (~0.6) [40]. Segmentation ambiguity and entropy variability in communication channels are major impediments that lead to decoding failures and restricted practical capacity [3]. Finally, computational optimizations induce significant overhead, with some reordering algorithms causing up to 69% performance degradation in processing speed [21].

3.6 Contextual Compatibility as a Research Direction

The findings surveyed above indicate a field in transition. Core problems remain unresolved, but the way researchers frame and address those problems has changed. The most visible change in the reviewed literature is the move from

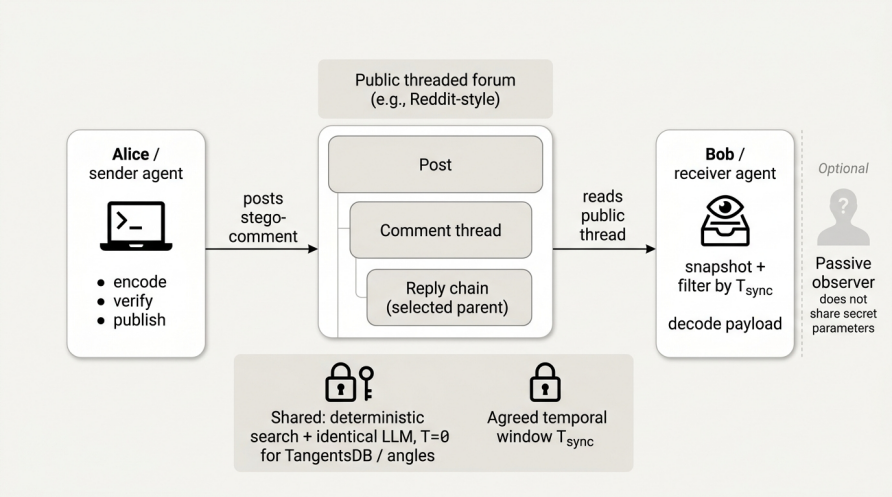

Fig. 2. Covert communication scenario: Alice and Bob use a public threaded channel; synchronization relies on shared operational parameters and a negotiated temporal window.

predominantly white-box, probability-driven methods toward a broader design space that includes black-box, context-aware, and robustness-oriented approaches. Earlier work depended heavily on local lexical or sampling manipulations, which are precise but difficult to scale and semantically fragile. More recent systems use LLMs to support full-text generation, rewriting, prompting, retrieval, and stylistic control, shifting the problem from isolated token selection toward the joint management of discourse, context, and transmission resilience. Whether this shift also produces more secure systems in practice remains insufficiently demonstrated by current evaluation methods.

The PSIC effect (Section 3.5) remains one of the central technical problems in the field. The reviewed studies collectively suggest that **contextual compatibility** can mitigate this conflict, although it does not resolve it. When systems align generated text with the topic, style, and discourse expectations of a target channel—drawing on the external knowledge sources surveyed in Section 3.4—the text can become more plausible to human readers without necessarily becoming more anomalous to statistical detectors. Framed this way, security is not only about distributional proximity to natural text; it also depends on whether the output functions as a plausible communicative act in its intended setting. This is the gap that the context-aware, selection-channel embedding proposed in Section 4 directly targets: rather than treating contextual fit as a secondary quality concern, it is used as the primary embedding domain, with reply targeting and angle selection carrying the payload while the generated comment itself functions as an ordinary, contextually grounded reply.

4 METHODOLOGY

This paper proposes a black-box steganographic technique that embeds information within the semantic architecture of a conversation. The methodology is predicated on a standard covert communication scenario involving Alice (the sender) and Bob (the receiver), communicating over a public social media platform. While this study utilizes Reddit-style data as a primary case study, the design is intended for threaded discussion environments; transfer to other platforms remains to be evaluated. Figure 2 sketches the end-to-end covert scenario and shared synchronization assumptions.

4.1 System Architecture

The core of the proposed framework centers on a multi-layer embedding process that leverages both the *spatial location* of the response and the *thematic subject* of the content as carriers for the secret payload. By mapping binary data to high-level conversational actions—specifically, the selection of a reply target and the selection of an *Angle*—the system carries the payload through a selection channel while preserving perceptual naturalness in the visible text. In the implemented workflow, the receiver reconstructs the same pre-sender context (thread snapshot + deterministic research + regenerated angles) and decodes the same selection indices to recover the payload. Figure 6 depicts the two embedding layers and their interaction with synthesis.

4.2 Operational Requirements

To achieve synchronization, both the sender and receiver must maintain access to:

- **Public Communication Channel:** Accounts on the target social media platform.
- **Shared Operational Parameters:** A deterministic search algorithm, post metadata, and predefined search constraints.
- **Deterministic Reconstruction Settings:** A synchronized model/backend configuration for deterministic stages (e.g., `temperature=0`) so both parties regenerate the same search terms and angle set from the same inputs.
- **Agreed Temporal Window:** A negotiated T_{sync} used to filter thread state so the receiver reconstructs the same parent-comment context seen by the sender.

4.3 Search Determinism & Synchronization Framework

Ensuring that both Alice and Bob operate on the same researched corpus is the cornerstone of reliable decoding. The implemented research stage is a reproducible preprocessing chain rather than an ad hoc web search. A search-term generator first extracts 12–20 unique queries from the post title, body, and URL using the workflow LLM at `temperature=0`. Each query is then executed through the Google Custom Search backend with a fixed request shape (`first=1`, `count=10`), duplicate links and PDF results are removed, and the surviving URLs are fetched in batches of three. Content extraction uses a crawl4ai-based adapter that normalizes URLs and extracts the primary text content. The resulting flat list of fetched page texts is stored as a search-results corpus, and the pipeline records term and corpus integrity hashes so the same corpus can be replayed by downstream stages. Figure 3 summarizes this deterministic research chain.

4.4 State Synchronization (Temporal Window)

To ensure that both Alice and Bob visualize an identical state of the conversation tree, they must adhere to a negotiated *Temporal Window Duration* (T_{sync}). This synchronization mechanism is operationalized to address the processing latency inherent in the sender's agent:

(1) **Sender Side:** Alice captures a snapshot of the discussion thread at time t_{start}, performs the multi-layer embedding, and subsequently publishes the generated stego-comment at $t_{publish}$.
(2) **Receiver Side:** Upon identifying the stego-contribution published at $t_{publish}$, Bob filters the thread's historical state to include only those comments established prior to $t_{publish} - T_{sync}$.

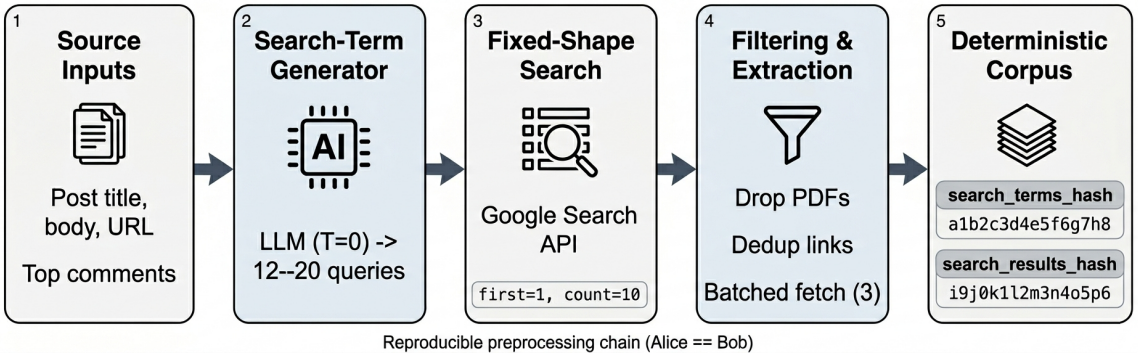

Fig. 3. Deterministic research stage: shared term generation, fixed search parameters, filtering, batched fetching, and integrity hashes for replayable corpora.

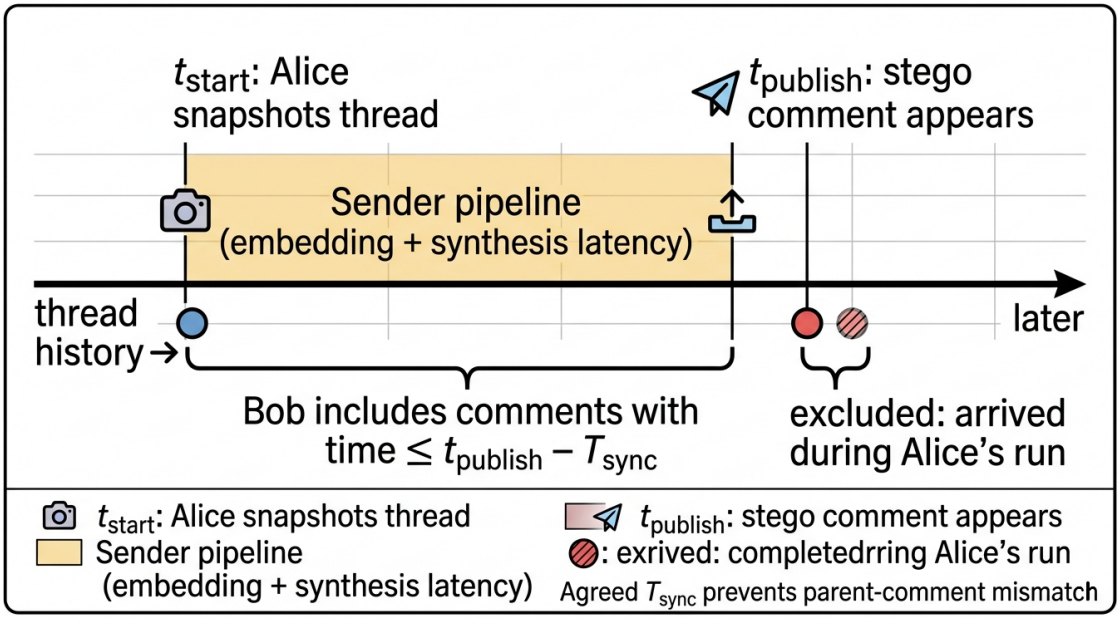

Fig. 4. Temporal synchronization: Bob's reconstructed thread state matches Alice's by agreeing on T_{sync} and filtering comments relative to $t_{publish}$.

The agreed-upon duration T_{sync} is an external synchronization requirement: the receiver must supply a pre-publication snapshot or apply the timestamp cutoff before reconstruction. The current receiver removes the sender comment and rebuilds from the supplied post, but does not itself enforce this timestamp cutoff; enforcement is an operational responsibility of the caller or acquisition layer. Figure 4 shows the intended temporal window relative to t_{start} and $t_{publish}$.

4.5 Payload Preparation & Compression

Before embedding, the payload passes through two reversible stages: an optional protection transform, then a dictionary-based compressor. Both stages are implemented once, in a single codec module that the sender and receiver both

Table 9. Token layout of the compressed payload stream. Every field has a fixed width given D, but decoding is strictly sequential: a token's start position depends on where the previous token ended, so the stream has no synchronization marker and cannot be resumed from an arbitrary offset.

Token	Field layout	Cost (bits)
Literal	`0` ‖ length ℓ (8 bits) ‖ UTF-8 bytes	$9 + 8b$
Reference	1 ‖ index j ($W(\lvert D\rvert)$ b) ‖ offset ($W(\lvert d_j\rvert)$ b) ‖ length ($W(L_{\max})$ b)	$1 + W(\lvert D\rvert) + W(\lvert d_j\rvert) + W(L_{\max})$

call directly — not two separate copies that could drift apart. Everything above that module (the sender and receiver pipelines) is limited to I/O, caching, and orchestration.

4.5.1 Protection Transform. The transform is chosen by configuration and recorded in the sender audit, so the receiver knows which inverse to apply. Three modes are implemented:

- `plain` (default): the payload is passed through unchanged.
- `hmac_xor_v1`: a keystream cipher. A fresh 16-byte nonce N and a pre-shared secret s generate a keystream $K = \mathrm{HMAC}_s(N\|c_0)\|\mathrm{HMAC}_s(N\|c_1)\|\cdots$ (SHA-256; c_i is the 4-byte big-endian counter i), and the UTF-8 payload is XOR-ed with K. A 16-byte truncated HMAC over the label `"swsec1"`, the nonce, and the ciphertext authenticates the result; the label keeps a tag from this mode from ever validating under `secure_compact_v2`. Nonce, ciphertext, and tag are each Base64-encoded (URL-safe) and joined with dots into one string.
- `secure_compact_v2`: the same cipher and tag construction under the label `"swsec2"`, except the payload is `zlib`-compressed before encryption, and the three raw fields are concatenated as bytes before Base64 encoding — producing one shorter blob instead of three dot-joined fields.

Tag verification runs in constant time, and a mismatch returns no payload rather than a corrupted one, so the receiver never emits an unauthenticated guess. This layer adds confidentiality on top of the covert channel; it is not what makes the channel covert.

4.5.2 Session Dictionary. The compressor operates against an ordered dictionary $D = (d_1, \ldots, d_{|D|})$ of text blocks in one fixed order: the post body, then each search-result text (full page text or snippet, whichever was retained), then every comment body in deterministic flattened order. This order matters because dictionary indices are positional: if the two parties assemble D differently, the same index silently points to different text on each side. A capacity profile can additionally cap and reorder these blocks — it ranks search-result and comment entries deterministically, then interleaves them round-robin — and is disabled by default, so both sides normally share the full, unranked source order described above.

4.5.3 Bit-Cost Optimal Parse. Let $W(m) = \lceil \log_2(m+1) \rceil$ for $m > 0$ and $W(m) = 0$ otherwise denote the fixed width of an unsigned field whose largest admissible value is m, and let $L_{\max} = \max_j |d_j|$. The payload is parsed into a sequence of tokens of the two kinds shown in Table 9.

A literal run is capped at 250 characters, which fixes its length field at $W(250) = 8$ bits. That length field counts characters, not bytes: the decoder reads 8-bit groups until exactly ℓ characters have been decoded, and b above is the resulting UTF-8 byte count. A reference is only ever generated for matches of three characters or more, since shorter matches are excluded before the dynamic program runs — their pointer overhead would exceed the literal bytes they would replace.

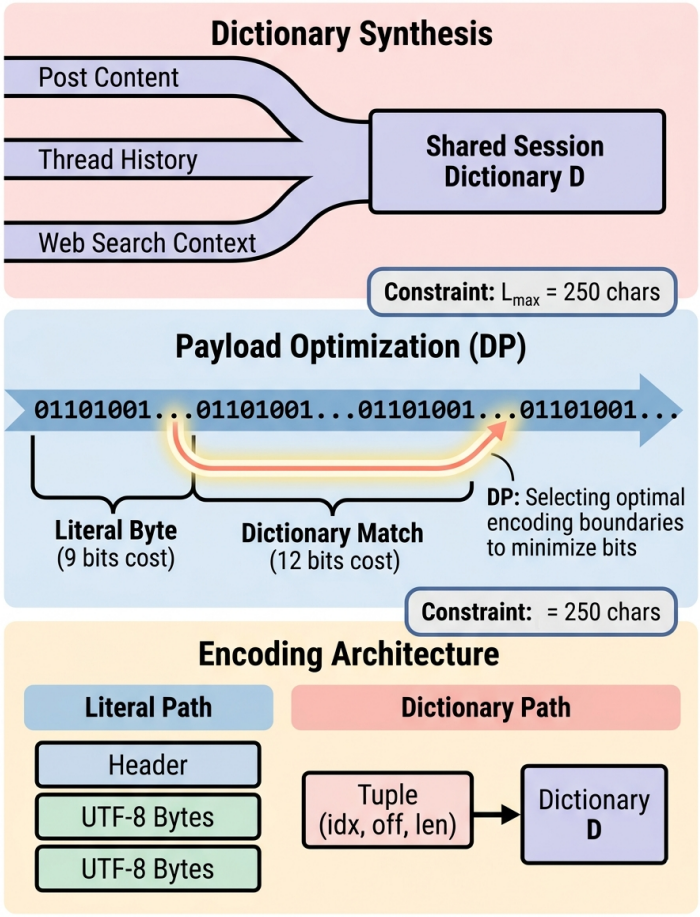

Fig. 5. Payload compression builds a session dictionary from thread and research text, then chooses literal versus dictionary-encoded segments to minimize total bit cost.

Given those costs the parse is chosen by a backward dynamic program over payload positions,

$$C(i) = \min_{t \in T(i)} \big(\mathrm{cost}(t) + C(i + |t|)\big), \qquad C(n) = 0,$$

where $T(i)$ contains every literal length up to the cap and, for each dictionary occurrence starting at i, only its longest contiguous extension — occurrences are located by greedy character-match extension rather than enumerated at every intermediate length. The dynamic program is therefore exact over that candidate set, not over every valid tokenization of the payload.

The emitted stream is one mode flag followed by the token sequence: flag `1` precedes the optimal parse, flag `0` precedes the raw UTF-8 bitstream. The compressor emits `0` whenever the parse is not strictly shorter than plain UTF-8, which bounds worst-case expansion at a single bit. Figure 5 outlines dictionary construction and the two encoding modes.

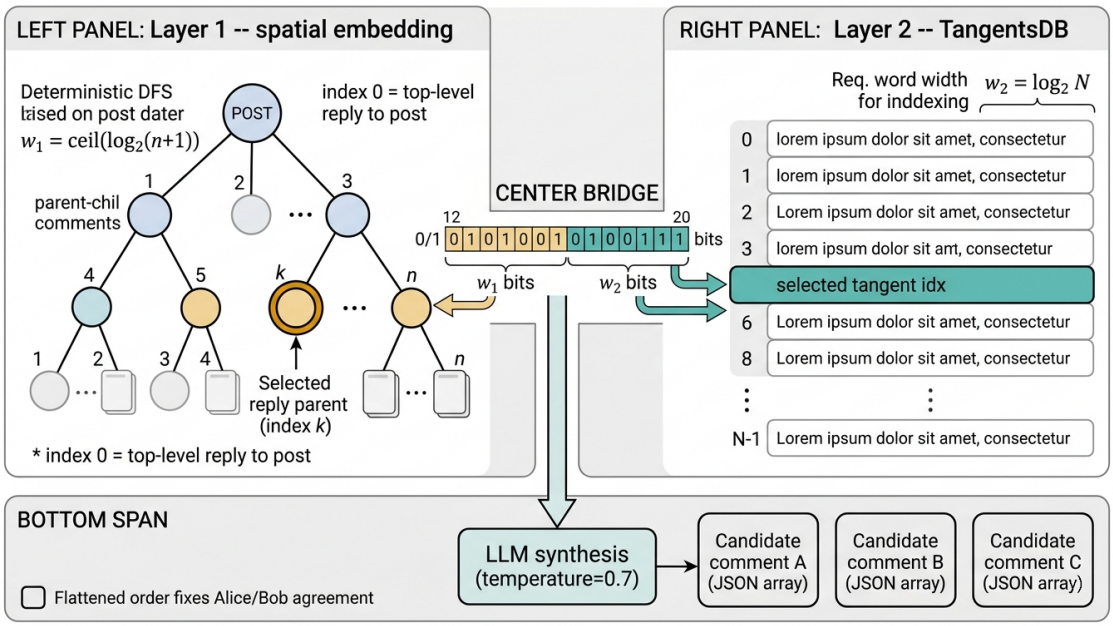

Fig. 6. Two-layer embedding: Layer 1 selects reply placement in a deterministically flattened thread; Layer 2 selects an Angle from a shared, deterministically reconstructed angle set; combined bits drive generative synthesis.

4.6 Two-Layer Semantic Embedding Architecture

Figure 6 provides the schematic overview of how the two embedding layers split the payload before synthesis.

4.6.1 Layer 1: Spatial Embedding (Reply Targeting). The primary embedding layer determines the *architectural placement* of the steganographic contribution.

(1) The system flattens the nested comment tree with a deterministic depth-first walk, so both sender and receiver number the same comments the same way.
(2) Each reply target gets a fixed-width index, but only $\lfloor \log_2 C \rfloor$ bits of it are guaranteed recoverable, where C is the number of selectable targets (Section 4.6.3 explains why the raw index width is not safe to use directly).
(3) Index 0 is a top-level reply to the post; any other index selects a specific comment, and the receiver reconstructs that comment's full parent chain by walking up the thread.

4.6.2 Layer 2: Intent-Based Embedding (Angle Selection). The secondary layer determines the *communicative intent.*

(1) **Angle Set Reconstruction:** Using the deterministic preprocessing chain (Section 4.3) and the filtered thread snapshot, both parties regenerate the same ordered list of candidate Angles, each described by a category label, a source quote, and a tangent field (the thematic direction).
(2) **Dynamic Bit-Mapping:** The flattened list is assigned sequential `idx` values, so the recoverable width is $\lfloor \log_2 A \rfloor$ bits for A available Angles — the same physical-versus-recoverable distinction from Section 4.6.3 applies here, with A in place of C.
(3) **Angle Selection:** The corresponding payload bits are mapped to a specific Angle index. During synthesis, the selected Angle is paired with the reply context and relevant research excerpts to produce a plausible cover reply.

4.6.3 Channel Widths and Bit Consumption Order. Both layers draw from the front of the same compressed stream produced in Section 4.5: Layer 1 consumes the first w_1 bits, Layer 2 the next w_2. Two width notions must be distinguished, and conflating them is the most common way to overstate this channel's capacity.

The *physical* widths are the field sizes actually written to the stream. For a thread with n comments there are $C = n+1$ reply targets, indexed 0 (a top-level reply to the post) through n, so $w_1 = W(n)$. For A flattened Angles, $w_2 = W(A-1)$. When the integer read from the bits exceeds the largest valid choice, the encoder reduces it modulo the choice count; this happens whenever the choice count is not a power of two. Generation therefore always succeeds, but several bit patterns then collapse onto the same observable selection, so those patterns cannot be inverted.

The *recoverable* widths exclude exactly that aliasing: $\lfloor \log_2 C \rfloor$ and $\lfloor \log_2 A \rfloor$. The lossless embedding path enforces this in two steps: it restricts each field's input to its recoverable width, then re-encodes that value at the full physical width before writing it. The result is a bits-to-selection map that is injective by construction. Capacity claims in Section 6 use these recoverable widths.

Consuming a prefix rather than a self-delimiting unit — one that states its own length up front — has two consequences. First, the very first bit Layer 1 reads is the compressor's mode flag, so the two layers depend on the compressor's output, not only on the thread and Angle set. Second, one post carries exactly $w_1 + w_2$ *physical* bits (Section 4.6.3); a longer payload leaves a surplus, which the encoder reports as unembedded rather than dropping silently. Recovering a payload larger than one post's capacity from a single visible comment therefore relies on sender-side metadata (audit-assisted recovery), and is reported as such in the evaluation. Blind recovery at scale is instead the job of the multi-frame protocol described next.

4.7 Multi-Frame Transport

To carry payloads beyond a single post's capacity, the sender splits the bitstream across several frames, where a frame is one reply comment placed under one post (a single post may host more than one frame — see below).

The payload is first protected as above and then compressed against an *empty* dictionary, which by the fallback rule of Section 4.5 always yields standard mode: a `0` flag followed by raw UTF-8 bits. Dictionary compression is deliberately not used here, because frames span several posts and no single session dictionary is shared by all of them.

The resulting bits are prefixed with a control header made of two Elias-gamma codes. An Elias-gamma code represents a positive integer as a unary length prefix followed by its binary value, so its size grows with the number it encodes instead of being fixed in advance. The header encodes the frame count F first, then the payload bit length, so the receiver can read off both quantities before reading any payload bits. A fixed-width header was rejected because the header is itself carried by the same scarce selection channel — it would need to be sized for the worst case even when F is small.

Frames are then assigned to slots: each post contributes up to a configured number of slots, and each slot's size is the recoverable capacity from Section 4.6.3. Sizing this is circular: the control header's length depends on the frame count F, but how many slots are needed depends on how large that header is. The planner resolves this by searching upward from $F = 1$, stopping at the smallest frame count whose first F slots have enough combined capacity for the stream built for that same F. The final frame is zero-padded to fill its slot. On the receiving side, the parser rejects any stream whose declared frame count disagrees with the observed one, or whose padding bits are not all zero — turning a desynchronized read into a clean failure rather than a plausible wrong payload.

The description above assumes a static Angle pool shared by all frames (the `post-level` sampler). Under the *context-weighted* sampler — the mode used to produce the results reported in Section 6 — the Angle pool instead depends on which parent comment was selected, so the planner works in two stages per frame: it resolves the parent-selection

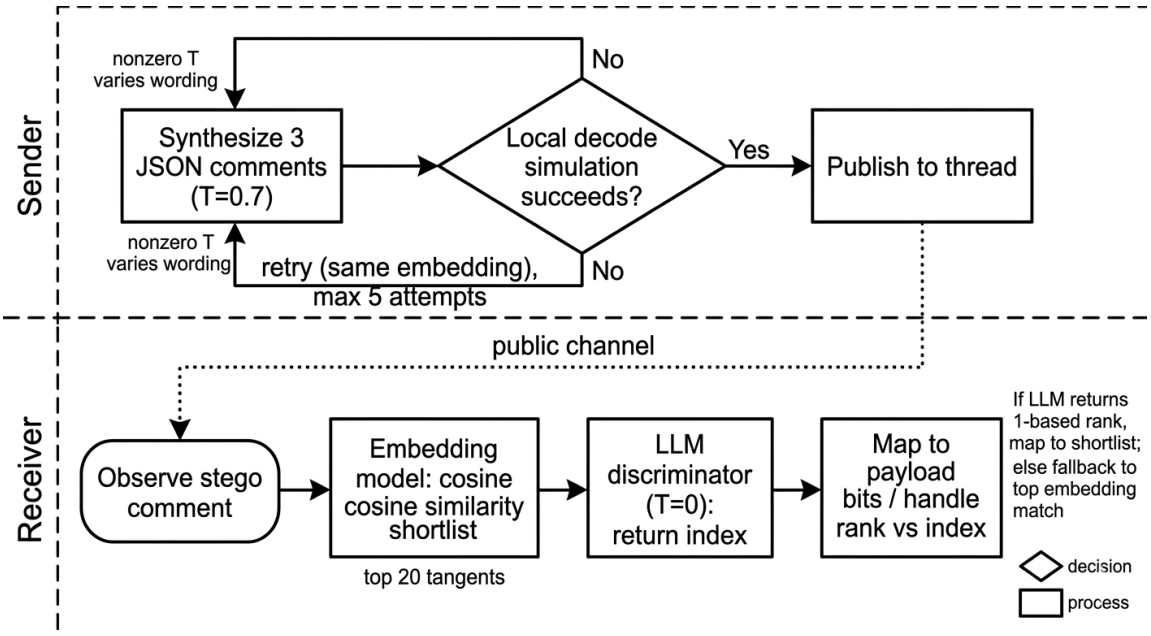

Fig. 7. Sender-side verification retries synthesis until decode succeeds; the receiver recovers Layer 2 via semantic shortlisting and a discriminative identification step.

bits first, regenerates the Angle set conditioned on that parent, and only then measures the Angle field width for the remaining bits.

4.8 Generative Synthesis & Verification

Following the selection of the target context and selected Angle, the sender initiates the cover text synthesis process.

- **Synthesis:** A configurable generative language model (run at non-zero temperature) produces a small fixed set of short, thread-appropriate reply candidates as a JSON list of non-empty strings. The prompt conditions on post metadata, the selected comment chain, the selected Angle (tangent), and relevant research excerpts.
- **Verification Loop:** Before the stego-text is committed to the public channel, the sender executes a local *Decoding Simulation* to verify that the generated text remains recoverable.
- **Recursive Refinement:** If the simulation fails to recover the intended Angle, the sender re-executes synthesis without changing the embedding parameters. The non-zero temperature ensures output variation; the retry loop is capped by a fixed attempt budget.

Figure 7 summarizes sender-side verification and receiver-side recovery.

4.9 Decoding & Intent Recovery

To recover the hidden intent (Layer 2), the receiver first reconstructs the pre-sender context deterministically, then decodes the Angle selection from the visible reply:

(1) **Context Reconstruction:** The receiver locates the candidate stego comment, removes it from the thread, applies the temporal cutoff ($t_{publish} - T_{sync}$), and reruns deterministic preprocessing (URL normalization, research, and angle generation) to reproduce the same ordered Angle list.

(2) **Semantic Filtering:** A semantic similarity model shortlists the top-N Angles whose text descriptions best match the observed stego reply.
(3) **Discriminative Identification:** A deterministic identification prompt is executed on the shortlisted candidates to return the most likely Angle index:
"Identify the specific Angle from the provided candidates... return the index only."

If the discriminator returns a 1-based rank rather than a raw index, the pipeline maps the rank back to the shortlist; otherwise it falls back to the top semantic match. The decode temperature and shortlist size are configurable, so this stage is not inherently deterministic. Recovery may therefore be pure selection-channel recovery only when the receiver reconstructs the same inputs and decoder contract, or audit-assisted when sender metadata is available. Combined with Layer 1 (reply targeting), the recovered indices reconstruct the selection bits.

5 IMPLEMENTATION

The practical implementation follows the same stage order as the workflow runner: data load, research, angle generation, stego synthesis, and decode. Each stage persists artifacts to disk so later stages can reuse the same corpus and hashes. Reproducibility is conditional on identical inputs, model/backend configuration, search responses or cached artifacts, and decoder settings; synthesis and semantic decoding remain configurable rather than universally deterministic.

5.1 Architectural Components

The system is operationalized through a series of deterministic pipeline stages. Core components include:

- **Data load stage:** Fetches unresolved URL content and replaces the post body with the extracted text.
- **Research stage:** Generates search terms from the post title, text, and URL using a deterministic LLM configuration (e.g., `temperature=0`), queries a fixed-shape search backend, removes duplicate links and PDFs, and fetches page text in bounded batches. The stage persists a flat search-results corpus together with reproducibility hashes.
- **Angle generation stage:** Flattens the corpus assembled from the post text, search-results corpus, and comments, then calls the angle runner to produce triplets consisting of a category, a source quote, and a tangent. Long inputs are split into 30k-character segments and batched up to 150k characters. If the backend path fails, the pipeline falls back to a direct LLM prompt that uses the same JSON schema.
- **Stego and decode stages:** Combine the compression layer, comment targeting, semantic shortlist, and verification loop. The encoder generates a small fixed number of candidate comments for the selected Angle, decodes them immediately, and retries synthesis if the selected Angle is not recoverable.

5.2 Codec Module and Sender–Receiver Contract

The bit layer described in Sections 4.5–4.7 is implemented once, as a module of pure functions shared by both pipelines. Dictionary construction, the field-width function W, the token grammar, and the physical-versus-recoverable width rules each have exactly one definition in that module: the sender and receiver pipelines call into it rather than reimplementing any of it, so encoding and decoding stay exact inverses given the same dictionary D. This is also what makes the contract testable offline: round-trip tests can drive the codec directly with fixtures, without a model or a network in the loop. It is why the pipelines themselves are restricted to I/O, caching, and orchestration.

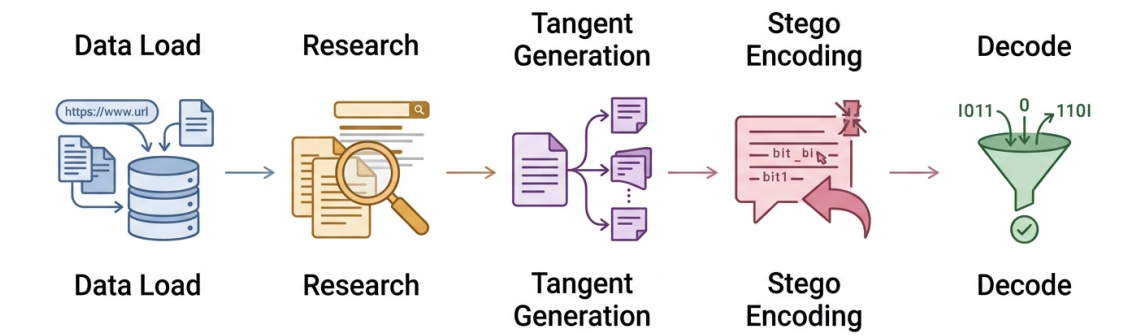

Fig. 8. Implementation stages and persisted artifacts align with the methodology: each stage is deterministic and replayable where required for sender–receiver parity.

Three configuration choices change the bit layer, so sender and receiver must agree on all of them: which payload transform is used (`plain`, `hmac_xor_v1`, or `secure_compact_v2`), the pre-shared secret required for `hmac_xor_v1` and `secure_compact_v2`, and whether the dictionary capacity profile is applied. The sender records the selected transform in the run audit, so the receiver reproduces it rather than having to infer it. The capacity profile is off by default because dictionary indices are positional: turning it on for only one side would leave the compressor and decompressor referring to different blocks under the same index. Figure 8 aligns these components with the staged implementation and on-disk artifacts.

5.3 Model Configuration

The implementation uses different model roles for different stages, but the exact model IDs are treated as configurable parameters:

- **Deterministic stages (term/angle generation):** Run at `temperature=0` to ensure reproducible reconstruction from the same inputs.
- **Stego synthesis:** Runs at non-zero temperature to generate diverse candidate replies for a fixed embedding choice.
- **Semantic matching and decode:** Uses a semantic similarity model for shortlist ranking, followed by a deterministic discriminative identification prompt (e.g., `temperature=0`) to select the final Angle index.

6 EVALUATION

This section reports a historical audited configuration comparison between the context-weighted method and the official zero-shot generative linguistic steganography (ZLG) baseline. It contains 304 decode-verified baseline pairs associated

Table 10. Post-clustered paired results over 100 source posts. Lower is better for GPT-2 perplexity. MATTR is shown on this run's 0–100 rescaling (55 · MATTR + 45). BERTScore is baseline-rescaled.

Metric	Our method	ZLG	Paired result
Recoverable capacity (bits/comment)	15.14	16.00	ZLG higher; Holm-adjusted sign-test $p < 0.001$
GPT-2 perplexity	68.15	53.70	ZLG lower; $p = 0.0001$
MATTR (rescaled 0–100)	99.36	98.45	Ours higher; $p = 0.0015$
Word count	22.4	17.3	Ours longer; $p < 0.001$
BERTScore Precision	0.083	0.154	ZLG higher; Holm-adjusted $p \approx 0.0045$
BERTScore F1	0.068	0.114	ZLG higher descriptively; Holm-adjusted $p = 0.12$

with 100 source-post clusters. Because repeated samples from the same post are not independent, comparisons are post-clustered; failed attempts remain in the attempt counts and are not silently discarded. The current research-state record does not treat this artifact as a clean, symmetric, decision-grade benchmark, so the results document observed trade-offs rather than a universal ranking.

6.1 Capacity Analysis

For one recoverable frame, the selection-channel capacity is $\lfloor \log_2 C \rfloor + \lfloor \log_2 A \rfloor$ bits, where C is the number of selectable reply targets and A is the number of flattened Angles. The floors are essential: when a choice count is not a power of two, the remaining index states are not counted as uniquely decodable payload states. Larger payloads require multiple independently recoverable frames. In this benchmark the Angle list was capped at 32 entries, so the angle component contributed at most 5 recoverable bits per frame.

6.2 Paired Benchmark Results

The full run attempted 627 samples with our method and produced 554 successes (88.4%). The baseline lane received those 554 successful outputs. In 94 cases our cover-sentence extraction failed before any request was issued, so the baseline was invoked on 460 of them and accepted 304 (66.1%) after its quality gate and prompt-leakage checks; all 304 baseline outputs were hide/reveal decode-verified. We report acceptance over the 460 invocations rather than over all 554 outputs, because the 94 extraction failures are a property of our harness and not of the baseline. Of the 156 genuine baseline failures, 133 were quality-gate rejections, 22 were prompt leakage, and one was a server fault. The project-side paired rows used audit-assisted compressed-bitstring recovery, so they validate selection/angle artifacts rather than demonstrating blind recovery of the complete compressed payload. The baseline used the server-matched EGS parameters (threshold = 0.01, temperature = 0.7, alpha = 1.0, max_bpw = 2) throughout.

The two methods therefore occupy different points in the trade-off space: ZLG leads on raw recoverable capacity and GPT-2 perplexity, while our method has slightly higher MATTR (lexical diversity on the run's rescaling) but produces longer replies. The capacity gap is narrow in this configuration because the angle pool was capped at 32; it should not be presented as a general upper bound on either system. On this run the former lexical-quality index reduces exactly to 55 · MATTR + 45, so the MATTR row is a diversity signal rather than an independent multi-factor quality win. BERTScore is baseline-rescaled; only Precision remains Holm-significant for ZLG ($p \approx 0.0045$), while F1 stays non-significant after correction. Reference-overlap scores (BLEU, ROUGE, BERTScore) also carry a thread-copy caveat: ZLG's cover sentence is drawn from the thread and can regurgitate text used as the overlap reference (about 5.9% of ZLG rows share a 5-gram with the BLEU reference versus 0.3% for ours). Mean BLEU gaps are therefore outlier-driven; medians are closer (1.46 versus 1.59). This benchmark does not establish a security advantage for either method.

6.3 Failure Analysis and Recovery

Our method's 73 failed attempts consisted of 70 sender/receiver angle mismatches and 3 generation failures. In the baseline lane, the 250 rejected attempts consisted of 134 quality-gate failures, 94 failures caused by insufficient cover sentences, and 22 prompt-leakage detections. These categories describe attempted-output reliability. They do not prove recovery of the full protected payload for the project arm, because the paired artifacts record audit-assisted compressed bits.

6.4 Sampling Caveats and Security Scope

Only 244 of the 304 our-method stegotexts were textually unique. The run used 167 researched posts, with 35 posts contributing the maximum ten samples; this reuse and the resulting dependence are disclosed rather than treated as independent trials. The 94 baseline cover-extraction failures were concentrated in 21 posts, so their aggregate count is not 94 independent observations.

Search quotas limited the supply of fresh posts, and post reuse produced 244 unique project texts among 304 rows. The comparison uses one angle-capacity configuration and one baseline service configuration, so it should be interpreted as an audited configuration comparison, not a universal ranking. Exploratory historical analyses found surface separability and recurring template signals; this benchmark does not establish resistance to an adaptive steganalyst or editing attacks. A blind, dedicated steganalysis and robustness evaluation remains future work.

7 CONCLUSION

This research introduces a context-aware semantic steganography framework for threaded discussions that redefines the embedding domain as a *selection channel*. Payload bits select a reply target and an Angle, while an LLM realizes the selected choice as visible text. The design avoids token-level payload insertion and supports black-box LLM access, but its recoverable capacity is limited by the available comment and Angle states.

The historical audited run attempted 627 samples with our method and obtained 554 successes; the baseline was invoked on 460 of these and 304 of its outputs were hide/reveal decode-verified. Across 100 post-clustered source posts, ZLG had higher recoverable capacity (16.00 versus 15.14 bits/comment) and lower GPT-2 perplexity (53.70 versus 68.15), whereas our method had slightly higher MATTR on this run's 0–100 rescaling (99.36 versus 98.45) and longer replies. The project-side rows used audit-assisted compressed-bitstring recovery, and the source posts were reused, so these results show a configuration trade-off rather than current decision-grade evidence or blind end-to-end superiority. Future work should establish a clean symmetric manifest, rerun blind decoding, reduce dependence on reused posts and exhausted search quotas, and test dedicated steganalysis and editing attacks.

REFERENCES

[1] Jipeng Qiang, Shiyu Zhu, Yun Li, Yi Zhu, Yunhao Yuan, and Xindong Wu. Natural language watermarking via paraphraser-based lexical substitution. *Artificial Intelligence*, 317:103859, 2023.

[2] Zhong-Liang Yang, Si-Yu Zhang, Yu-Ting Hu, Zhi-Wen Hu, and Yong-Feng Huang. Vae-stega: linguistic steganography based on variational auto-encoder. *IEEE Transactions on Information Forensics and Security*, 16:880–895, 2020.

[3] Gabriel Kaptchuk, Tushar M Jois, Matthew Green, and Aviel D Rubin. Meteor: Cryptographically secure steganography for realistic distributions. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, pages 1529–1548, Virtual Event, Republic of Korea, 2021. ACM.

[4] Yihao Wang, Ru Zhang, Yifan Tang, and Jianyi Liu. State-of-the-art advances of deep-learning linguistic steganalysis research. In *2023 International Conference on Data, Information and Computing Science (CDICS)*, page 20–24. IEEE, December 2023. doi: 10.1109/cdics61497.2023.00014. URL http://dx.doi.org/10.1109/CDICS61497.2023.00014.

[5] Fanxiao Li, Sixing Wu, Jiong Yu, Shuoxin Wang, BingBing Song, Renyang Liu, Haoseng Lai, and Wei Zhou. Rewriting-stego: generating natural and controllable steganographic text with pre-trained language model. In *International Conference on Database Systems for Advanced Applications*, pages 617–626. Springer, 2023.

[6] Ke Lin, Yiyang Luo, Zijian Zhang, and Ping Luo. Zero-shot generative linguistic steganography. *arXiv preprint arXiv:2403.10856*, 2024.

[7] Jiaxuan Wu, Zhengxian Wu, Yiming Xue, Juan Wen, and Wanli Peng. Generative text steganography with large language model. In *Proceedings of the 32nd ACM International Conference on Multimedia*, pages 10345–10353, Melbourne, Australia, 2024. ACM.

[8] Guorui Liao, Jinshuai Yang, Kaiyi Pang, and Yongfeng Huang. Co-stega: Collaborative linguistic steganography for the low capacity challenge in social media. In *Proceedings of the 2024 ACM Workshop on Information Hiding and Multimedia Security*, pages 7–12, Baiona, Spain, 2024. ACM.

[9] Huili Wang, Zhongliang Yang, Jinshuai Yang, Yue Gao, and Yongfeng Huang. Hi-stega: A hierarchical linguistic steganography framework combining retrieval and generation. In *International Conference on Neural Information Processing*, pages 41–54. Springer, 2023.

[10] Ruifan Zhang, Jianyi Liu, and Ru Zhang. Controllable semantic linguistic steganography via summarization generation. In *ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 4560–4564. IEEE, 2024.

[11] Gustavus J Simmons. The prisoners' problem and the subliminal channel. In *Advances in Cryptology: Proceedings of Crypto 83*, pages 51–67, Boston, MA, 1984. Springer.

[12] Siyu Zhang, Zhongliang Yang, Jinshuai Yang, and Yongfeng Huang. Provably secure generative linguistic steganography. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, pages 3046–3055, 2021.

[13] Changhao Ding, Zhangjie Fu, Zhongliang Yang, Qi Yu, Daqiu Li, and Yongfeng Huang. Context-aware linguistic steganography model based on neural machine translation. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 32:868–878, 2023.

[14] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[15] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019.

[16] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33:1877–1901, 2020.

[17] Sébastien Bubeck, Varun Chandrasekaran, Ronen Eldan, Johannes Gehrke, Eric Horvitz, Ece Kamar, Peter Lee, Yin Tat Lee, Yuanzhi Li, Scott Lundberg, et al. Sparks of artificial general intelligence: Early experiments with GPT-4. *arXiv preprint arXiv:2303.12712*, 2023.

[18] Emily Reif, Daphne Ippolito, Ann Yuan, Andy Coenen, Chris Callison-Burch, and Jason Wei. A recipe for arbitrary text style transfer with large language models. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 837–848, 2022.

[19] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

[20] Aiyuan Yang, Bin Xiao, Binyuan Wang, Binxin Zhang, Ce Bian, Chao Yin, Chenxu Lv, Da Pan, Dian Wang, Dong Yan, et al. Baichuan 2: Open large-scale language models. *arXiv preprint arXiv:2309.10305*, 2023.

[21] Jinyang Ding, Kejiang Chen, Yaofei Wang, Na Zhao, Weiming Zhang, and Nenghai Yu. Discop: Provably secure steganography in practice based on distribution copies. In *2023 IEEE Symposium on Security and Privacy (SP)*, pages 2238–2255, San Francisco, CA, USA, 2023. IEEE.

[22] Yuang Qi, Kejiang Chen, Kai Zeng, Weiming Zhang, and Nenghai Yu. Provably secure disambiguating neural linguistic steganography. *IEEE Transactions on Dependable and Secure Computing*, 2024. Early Access.

[23] Mohammed Abdul Majeed, Rossilawati Sulaiman, Zarina Shukur, and Mohammad Kamrul Hasan. A review on text steganography techniques. *Mathematics*, 9(21), 2021. ISSN 2227-7390. doi: 10.3390/math9212829. URL https://www.mdpi.com/2227-7390/9/21/2829.

[24] De Rosal Ignatius Moses Setiadi, Sudipta Kr Ghosal, and Aditya Kumar Sahu. Ai-powered steganography: Advances in image, linguistic, and 3d mesh data hiding – a survey. *Journal of Future Artificial Intelligence and Technologies*, 2(1):1–23, Apr. 2025. doi: 10.62411/faith.3048-3719-76. URL https://faith.futuretechsci.org/index.php/FAITH/article/view/76.

[25] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI Blog*, 2019. URL https://d4mucfpksywv.cloudfront.net/better-language-models/language_models_are_unsupervised_multitask_learners.pdf.

[26] Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, et al. The rise and potential of large language model based agents: A survey. *Science China Information Sciences*, 68(2):121101, 2025.

[27] Jianfei Xiao, Yancan Chen, Yimin Ou, Hanyi Yu, Kai Shu, and Yiyong Xiao. Baichuan2-sum: Instruction finetune baichuan2-7b model for dialogue summarization. In *2024 International Joint Conference on Neural Networks (IJCNN)*, pages 1–8. IEEE, 2024.

[28] Siyu Zhang, Zhongliang Yang, Jinshuai Yang, and Yongfeng Huang. Linguistic steganography: From symbolic space to semantic space. *IEEE Signal Processing Letters*, 28:11–15, 2020.

[29] Sebastian Zillien, Tobias Schmidbauer, Mario Kubek, Joerg Keller, and Steffen Wendzel. Look what's there! utilizing the internet's existing data for censorship circumvention with oppression. In *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security*, pages 80–95, New York, NY, USA, 2024. ACM.

[30] Martin Steinebach. Natural language steganography by chatgpt. In *Proceedings of the 19th International Conference on Availability, Reliability and Security*, pages 1–9. ACM, 2024.

[31] Oleksandr Kuznetsov, Kyrylo Chernov, Aigul Shaikhanova, Kainizhamal Iklassova, and Dinara Kozhakhmetova. DeepStego: Privacy-preserving natural language steganography using large language models and advanced neural architectures. *Computers*, 14(5):165, 2025.

[32] Kamil Woźniak, Marek R. Ogiela, and Lidia Ogiela. Multi-criteria linguistic optimization for covert communication in secure LLM-based steganography. *Applied Soft Computing*, 185:113960, 2025.

[33] Hao Shi, Wenpu Guo, and Shaoyuan Gao. Emotionally controllable text steganography based on large language model and named entity. *Technologies*, 13(7):264, 2025.

[34] Shuo Lin, Zhenyang Shen, Xiang Li, Jiacheng Fan, and Sixing Wu. Position-agnostic probabilistic generation for robust steganographic text. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*, pages 4950–4954. ACM, 2025.

[35] Yingquan Chen, Qianmu Li, Xiaocong Wu, and Zijian Ying. Research on making two models based on the generative linguistic steganography for securing linguistic steganographic texts from active attacks. *Symmetry*, 17(9):1416, 2025.

[36] Kaiyi Pang, Minhao Bai, Jinshuai Yang, Huili Wang, Minghu Jiang, and Yongfeng Huang. Fremax: A simple method towards truly secure generative linguistic steganography. In *ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 4755–4759. IEEE, 2024.

[37] Yihao Li, Ru Zhang, Jianyi Liu, and Qi Lei. A semantic controllable long text steganography framework based on llm prompt engineering and knowledge graph. *IEEE Signal Processing Letters*, 2024.

[38] Xiaoyan Zheng, Yurun Fang, and Hanzhou Wu. General framework for reversible data hiding in texts based on masked language modeling. In *2022 IEEE 24th International Workshop on Multimedia Signal Processing (MMSP)*, pages 1–6. IEEE, 2022.

[39] Changhao Ding, Zhangjie Fu, Qi Yu, Fan Wang, and Xianyi Chen. Joint linguistic steganography with bert masked language model and graph attention network. *IEEE Transactions on Cognitive and Developmental Systems*, 16(2):772–781, 2023.

[40] Zhe Ji, Qiansiqi Hu, Yicheng Zheng, Liyao Xiang, and Xinbing Wang. A principled approach to natural language watermarking. In *Proceedings of the 32nd ACM International Conference on Multimedia*, pages 2908–2916. ACM, 2024.

[41] Lingyun Xiang, Jiali Xia, Yangfan Liu, and Yan Gui. Cpg-ls: Causal perception guided linguistic steganography. *IEEE Signal Processing Letters*, 30: 1762–1766, 2023.

[42] Biao Yi, Hanzhou Wu, Guorui Feng, and Xinpeng Zhang. Alisa: Acrostic linguistic steganography based on bert and gibbs sampling. *IEEE Signal Processing Letters*, 29:687–691, 2022.

[43] Travis Munyer, Abdullah All Tanvir, Arjon Das, and Xin Zhong. Deeptextmark: a deep learning-driven text watermarking approach for identifying large language model generated text. *Ieee Access*, 12:40508–40520, 2024.

[44] Fanxiao Li, Ping Wei, Tingchao Fu, Yu Lin, and Wei Zhou. Imperceptible text steganography based on group chat. In *2024 IEEE International Conference on Multimedia and Expo (ICME)*, pages 1–6. IEEE, 2024.

[45] Zhenyu Xu, Ruoyu Xu, and Victor S Sheng. Beyond binary classification: Customizable text watermark on large language models. In *2024 International Joint Conference on Neural Networks (IJCNN)*, pages 1–8. IEEE, 2024.

[46] Jifei Hao, Jipeng Qiang, Yi Zhu, Yun Li, Yunhao Yuan, Xiaocheng Hu, and Xiaoye Ouyang. Robust and semantic-faithful post-hoc watermarking of text generated by black-box language models. *Frontiers of Computer Science*, 19(9):199357, 2025.

[47] Jianxin Wang, Xiangze Chang, Chaoen Xiao, and Lei Zhang. A contrastive semantic watermarking framework for large language models. *Symmetry*, 17(7):1124, 2025.

[48] Fred Jelinek, Robert L Mercer, Lalit R Bahl, and James K Baker. Perplexity—a measure of the difficulty of speech recognition tasks. *The Journal of the Acoustical Society of America*, 62(S1):S63–S63, 1977.

[49] Yequan Wang, Jiawen Deng, Aixin Sun, and Xuying Meng. Perplexity from plm is unreliable for evaluating text quality. *arXiv preprint arXiv:2210.05892*, 2022.

[50] Lizhe Fang, Yifei Wang, Zhaoyang Liu, Chenheng Zhang, Stefanie Jegelka, Jinyang Gao, Bolin Ding, and Yisen Wang. What is wrong with perplexity for long-context language modeling? In *International Conference on Learning Representations*, volume 2025, pages 94541–94563, 2025.

[51] Abby Morgan. Perplexity for LLM evaluation. Comet ML Blog, November 2024. URL https://www.comet.com/site/blog/perplexity-for-llm-evaluation/. Accessed: 2025-12-31.

[52] S. Kullback and R. A. Leibler. On information and sufficiency. *The Annals of Mathematical Statistics*, 22(1):79–86, 1951. ISSN 00034851. URL http://www.jstor.org/stable/2236703.

[53] Christian Cachin. An information-theoretic model for steganography. In David Aucsmith, editor, *Information Hiding*, pages 306–318, Berlin, Heidelberg, 1998. Springer Berlin Heidelberg. ISBN 978-3-540-49380-8.

[54] Emily M Bender, Timnit Gebru, Angelina McMillan-Major, and Shmargaret Shmitchell. On the dangers of stochastic parrots: Can language models be too big? In *Proceedings of the 2021 ACM conference on fairness, accountability, and transparency*, pages 610–623, Virtual Event, Canada, 2021. ACM.